



1. Problem Title & Link

Title: LeetCode 217 : Contains Duplicate

Link: [LeetCode 217 - Contains Duplicate](#)

2. Problem Statement (Short Summary)

You are given an integer array:

nums[]

Return:

true

if any value appears at least twice.

Return:

false

if every element is distinct.

In simple terms:

Check whether the array contains repeated values.

3. Examples (Input → Output)

Example 1

Input:

nums=[1,2,3,1]

Output:

true

Explanation:

1 appears twice

Example 2

Input:

nums=[1,2,3,4]

Output:

false

Explanation:

All elements are unique.

Example 3

Input:

nums=[1,1,1,3,3,4,3,2,4,2]

Output:



true

4. Constraints

$1 \leq \text{nums.length} \leq 10^5$

$-10^9 \leq \text{nums}[i] \leq 10^9$

Special restrictions:

- Need efficient lookup
- Avoid nested loops

5. Core Concept (Pattern / Topic)

HashSet

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every element:

1. Compare with all remaining elements
2. If duplicate found $\rightarrow$ return true

Time:

$O(n^2)$

Too slow.

Optimized Thinking

Observation:

HashSet allows:

Insert $\rightarrow O(1)$

Search $\rightarrow O(1)$

Algorithm:

1. Traverse array
2. Check whether element already exists in set
3. If yes:

return true

4. Otherwise add element

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

[1,2,3,1]



Processing:

Set={}

1 → add

Set={1}

2 → add

Set={1,2}

3 → add

Set={1,2,3}

1 → already exists

Return true

8. Pseudocode (Language Independent)

create set

for num in nums:

 if num exists in set:

 return true

 add num to set

return false

9. Code Implementation

Python

class Solution:

 def containsDuplicate(self, nums):



```
seen=set()

for num in nums:

    if num in seen:
        return True

    seen.add(num)

return False
```

Java

```
class Solution {

    public boolean containsDuplicate(
        int[] nums) {

        Set<Integer> set=
            new HashSet<>();

        for(int num:nums){

            if(set.contains(num))
                return true;

            set.add(num);
        }

        return false;
    }
}
```

10. Time & Space Complexity

Time Complexity:

$O(n)$

Reason:



Array traversed once.

Space Complexity:

$O(n)$

Reason:

Set may store all elements.

11. Common Mistakes / Edge Cases

- Using nested loops

Wrong:

for i

for j

Causes:

$O(n^2)$

- Forgetting negative numbers

[-1,-2,-3,-1]

- Missing single element case

[5]

Output:

false

- Using list instead of set

Searching in list:

$O(n)$

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[1,2,3,1]

Index	Current Number	Set	Action
0	1	{}	Add
1	2	{1}	Add
2	3	{1,2}	Add
3	1	{1,2,3}	Found duplicate

Output:



true

13. Common Use Cases (Real-Life / Interview)

Duplicate user detection

Fraud transaction detection

Database uniqueness checking

Caching systems

14. Common Traps (Important!)

1. Using array for lookup
2. Using nested loops
3. Forgetting HashSet lookup advantage
4. Missing duplicate detection early

15. Builds To (Related LeetCode Problems)

Progression:

- LC 217 → Contains Duplicate
- LC 219 → Contains Duplicate II
- LC 220 → Contains Duplicate III
- LC 128 → Longest Consecutive Sequence
- LC 1 → Two Sum

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Too slow
Sorting	$O(n \log n)$	$O(1)$	Compare adjacent values
HashSet	$O(n)$	$O(n)$	Optimal

17. Why This Solution Works (Short Intuition)

A HashSet stores previously seen values. Whenever a number appears again, we immediately know it is a duplicate because lookup in a HashSet is very fast.

18. Variations / Follow-Up Questions

1. Find duplicate value instead of true/false
2. Return frequency of duplicates



3. Find duplicate within distance k
4. Count distinct elements
5. Find first repeating element
- 6.